

Lexer

Traductor de Lenguaje a Binario

Sistemas Operativos de Base 1

Universidad Autónoma de Tamaulipas

Facultad de Ingeniería Tampico

Ingeniería en Sistemas Computacionales

8 ° M

Docente: Dante Adolfo Muñoz Quintero

Integrantes:
Rosales García Ricardo Javier

Contenido

1.	Introducción	3
2.	Objetivos	4
	Objetivo General.....	4
	Objetivos Específicos	4
3.	Especificación del Lenguaje	5
3.1	Nombre y Propósito	5
3.2	Ejemplos de Código	5
3.3	Catálogo de Tokens	6
4.	Implementación.....	7
5.	Demostración Final	9
6.	Conclusiones.....	14
7.	Referencias.....	14

1. Introducción

En la actualidad no muchas de las personas en la actualidad conocen como se envía un mensaje por la red o como es que un simple clic puede desencadenar una serie de acciones tan complejas dependiendo de las tareas que se buscan realizar, este proyecto busca generar una cierta conciencia acerca de como se puede interpretar una serie de textos de una forma en la que las maquinas generalmente se comunican.

Se busca la necesidad de garantizar integridad en la codificación de instrucciones que pueden viajar entre sistemas y de exponer claramente cómo un texto fuente se tokeniza, parsea y convierte a una forma binaria, facilitando la depuración de conceptos de compiladores y redes de comunicación.

Este proyecto fue implementado en Python 3.11 usando herramientas de apoyo como ANTLR y ciertos modelos de IA debido a la complejidad del trabajo que se busca realizar.

ANTLR es una herramienta muy utilizada, con más de 1000 usuarios registrados del ámbito industrial y académico en 37 países. Se ha adaptado a numerosos sistemas populares, como PC, Macintosh y diversas plataformas UNIX; además, como resultado de una de nuestras colaboraciones con el sector industrial, se ha desarrollado una interfaz comercial en C++.

La traducción de lenguajes informáticos se ha convertido en una tarea habitual. Aunque siguen desarrollándose compiladores y herramientas para los lenguajes informáticos tradicionales (como C o Java). Los programadores crean traductores para formatos de bases de datos, archivos de datos y archivos de procesamiento de texto. ANTLR está diseñado para gestionar todas tus tareas de traducción.

2. Objetivos

Objetivo General

Diseñar e implementar un lenguaje didáctico y su cadena de procesamiento (lexer, parser y verificación binaria) para que usuarios comprendan y validen cómo un texto fuente se tokeniza, parsea y se representa en bits, facilitando la detección de errores léxicos y la trazabilidad de la salida para escenarios de transmisión de datos.

Objetivos Específicos

1. Especificar formalmente la sintaxis del lenguaje mediante una gramática ANTLR que cubra sentencias if, then, expresiones aritméticas, comparadores y reglas léxicas de error.
2. Implementar el lexer y parser en Python usando los artefactos generados por ANTLR
3. Detectar y reportar errores léxicos comunes automáticamente como símbolos desconocidos, operadores sin operando, etc. con línea y columna.
4. Generar y validar la representación binaria del código fuente para comprobar integridad y preparar la salida para transmisión.

3. Especificación del Lenguaje

3.1 Nombre y Propósito

La representación binaria del código procesado por este proyecto responde a una necesidad práctica: todo dato que circula por redes de comunicación se transmite finalmente como secuencias de bits y busca generar conocimiento de cómo se llega a ese proceso.

3.2 Ejemplos de Código

Ejemplo 1. Token

```
def token_name_safe(lexer, token):  
    if lexer is None:  
        return None  
    names = lexer.symbolicNames  
    ttype = token.type  
    if ttype is None:  
        return "UNKNOWN"  
    if 0 <= ttype < len(names) and names[ttype] is not None:  
        return names[ttype]  
    if ttype == Token.EOF:  
        return "EOF"  
    return "UNKNOWN"
```

Ejemplo 2. Traducción a binario

```
def text_to_binary_lines(text):  
    lines = text.splitlines()  
    binary_lines = []  
    for line in lines:  
        if line == "":  
            binary_lines.append("")  
            continue  
        bytes_seq = line.encode('utf-8')  
        binary_lines.append(' '.join(format(b, '08b') for b in bytes_seq))  
    if text.endswith('\n'):  
        binary_lines.append("")  
    return binary_lines
```

Ejemplo 3. Revisión de Operadores

```
def is_operator_name(name):
    return name in {"PLUS", "MINUS", "MUL", "DIV", "MOD", "EQ", "NEQ", "GE", "LE", "GT", "LT", "ASSIGN"}
```

3.3 Catálogo de Tokens

Token	Patrón	Descripción	Ejemplo
IF	if	Palabra reservada inicio de sentencia condicional	if
THEN	then	Palabra reservada separador de condición y asignación	then
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	Nombre de variable o etiqueta válida	result, _tmp1
NUMBER	[0-9]+(\.[0-9]+)?	Literal numérico entero o decimal	123, 3.14
ASSIGN	=	Operador de asignación	=
DELIMITER	;	Final de sentencia	;
PLUS	+	Suma	+
MINUS	-	Resta / unario -	-, a - b
MUL	*	Multipliación	*
DIV	/	División	/
MOD	%	Módulo	%
EQ	==	Igualdad	==
NEQ	!= (o != opcional)	Desigualdad	!=
GE	>=	Mayor o igual	>=
LE	<=	Menor o igual	<=
GT	>	Mayor que	>
LT	<	Menor que	<

LPAREN	(	Paréntesis izquierdo	(
RPAREN	)	Paréntesis derecho	)
UNKNOWN	.	Cualquier símbolo no reconocido (error léxico)	@, #

4. Implementación

A continuación, se presenta el código fuente de los módulos del analizador léxico.

4.1 Definición de operadores

```
// Operadores
PLUS      : '+' ;
MINUS     : '-' ;
MUL       : '*' ;
DIV       : '/' ;
MOD       : '%' ;
```

4.2 Definición de comparadores

```
// Comparadores
EQ      : '==' ;
NEQ     : '!=' | '!' [ \t\r\n]* '=' ;
GE      : '>=' ;
LE      : '<=' ;
GT      : '>' ;
LT      : '<' ;
```

4.3 Reglas

```
statement
    : IF expr THEN IDENTIFIER ASSIGN expr DELIMITER
    ;

expr
    : relationalExpr
    ;

relationalExpr
    : additiveExpr ( (GT | LT | GE | LE | EQ | NEQ) additiveExpr )?
    ;

additiveExpr
    : multiplicativeExpr ( (PLUS | MINUS) multiplicativeExpr )*
    ;

multiplicativeExpr
    : unaryExpr ( (MUL | DIV | MOD) unaryExpr )*
    ;

unaryExpr
    : MINUS unaryExpr
    | primary
    ;
```

4.4 Lexer Básico

```
# Detecciones léxicas básicas
lexical_errors = []
for token in tokens:
    name = token_name_safe(lexer, token)
    if name == "INVALID_OP":
        lexical_errors.append((token.line, token.column, token.text, "Invalid operator sequence"))
    elif name == "INVALID_ID":
        lexical_errors.append((token.line, token.column, token.text, "Invalid identifier"))
    elif name == "UNKNOWN":
        lexical_errors.append((token.line, token.column, token.text, "Unknown symbol"))
```


5. Demostración Final

5.1 Ejecución de programa valido

Archivo de entrada

```
1  if x + y * (z - 2) >= 100 then result = x * y + z / 2;  
2  if (a - 3) * (b + 4) < c + 10 then out = (a + b) * (c - 1);|
```

Salida en consola

```
Tokens  
1:0 -> (IF, "if")  
1:3 -> (IDENTIFIER, "x")  
1:5 -> (PLUS, "+")  
1:7 -> (IDENTIFIER, "y")  
1:9 -> (MUL, "*")  
1:11 -> (LPAREN, "(")  
1:12 -> (IDENTIFIER, "z")  
1:14 -> (MINUS, "-")  
1:16 -> (NUMBER, "2")  
1:17 -> (RPAREN, ")")  
1:19 -> (GE, ">=")  
1:22 -> (NUMBER, "100")  
1:26 -> (THEN, "then")  
1:31 -> (IDENTIFIER, "result")  
1:38 -> (ASSIGN, "=")  
1:40 -> (IDENTIFIER, "x")  
1:42 -> (MUL, "*")  
1:44 -> (IDENTIFIER, "y")  
1:46 -> (PLUS, "+")  
1:48 -> (IDENTIFIER, "z")  
1:50 -> (DIV, "/")  
1:52 -> (NUMBER, "2")  
1:53 -> (DELIMITER, ";")
```

```
2:0 -> (IF, "if")
2:3 -> (LPAREN, "(")
2:4 -> (IDENTIFIER, "a")
2:6 -> (MINUS, "-")
2:8 -> (NUMBER, "3")
2:9 -> (RPAREN, ")")
2:11 -> (MUL, "*")
2:13 -> (LPAREN, "(")
2:14 -> (IDENTIFIER, "b")
2:16 -> (PLUS, "+")
2:39 -> (ASSIGN, "=")
2:41 -> (LPAREN, "(")
2:42 -> (IDENTIFIER, "a")
2:44 -> (PLUS, "+")
2:46 -> (IDENTIFIER, "b")
2:47 -> (RPAREN, ")")
2:49 -> (MUL, "*")
2:51 -> (LPAREN, "(")
2:52 -> (IDENTIFIER, "c")
2:54 -> (MINUS, "-")
2:56 -> (NUMBER, "1")
2:57 -> (RPAREN, ")")
2:58 -> (DELIMITER, ";")

=== Parse OK ===
```

Traducción a Binario (parcial)

```
Line 1: 01101001 01100110 00100000 01111000  
01111110 00111101 00100000 00110001 00110000  
01000000 01111000 00100000 00101010 00100000
```

Ejecución con errores

```
1:0 -> (IF, "if")  
1:3 -> (IDENTIFIER, "x")  
1:5 -> (INVALID_OP, "++")  
1:8 -> (IDENTIFIER, "y")  
1:10 -> (THEN, "then")  
1:15 -> (IDENTIFIER, "a")  
1:17 -> (ASSIGN, "=")  
1:19 -> (IDENTIFIER, "b")  
1:20 -> (DELIMITER, ";")  
2:0 -> (IF, "if")  
2:3 -> (IDENTIFIER, "total")  
2:9 -> (INVALID_OP, "**")  
2:12 -> (NUMBER, "2")  
2:14 -> (GT, ">")  
2:16 -> (NUMBER, "100")  
2:20 -> (THEN, "then")  
2:25 -> (IDENTIFIER, "pow")  
2:29 -> (ASSIGN, "=")  
2:31 -> (IDENTIFIER, "total")  
2:37 -> (INVALID_OP, "**")  
2:40 -> (NUMBER, "2")  
2:41 -> (DELIMITER, ";")
```

```
3:0 -> (IF, "if")
3:3 -> (IDENTIFIER, "a")
3:5 -> (INVALID_OP, "&&")
3:7 -> (INVALID_OP, "||")
3:10 -> (IDENTIFIER, "b")
3:12 -> (THEN, "then")
3:17 -> (IDENTIFIER, "ok")
3:20 -> (ASSIGN, "=")
3:22 -> (NUMBER, "0")
3:23 -> (DELIMITER, ";")
4:0 -> (IF, "if")
4:3 -> (NUMBER, "12.34")
4:8 -> (UNKNOWN, ".")
4:9 -> (NUMBER, "56")
4:12 -> (GT, ">")
4:14 -> (NUMBER, "0")
4:16 -> (THEN, "then")
4:21 -> (IDENTIFIER, "badnum")
4:28 -> (ASSIGN, "=")
4:30 -> (NUMBER, "1")
4:31 -> (DELIMITER, ";")
5:0 -> (IF, "if")
```

```
5:3 -> (IDENTIFIER, "value")
5:9 -> (UNKNOWN, "$")
5:11 -> (NUMBER, "5")
5:13 -> (THEN, "then")
5:18 -> (IDENTIFIER, "v")
5:20 -> (ASSIGN, "=")
5:22 -> (IDENTIFIER, "value")
5:27 -> (DELIMITER, ";")
```

Lista de errores

```
Error List
Line 1, Column 5: Invalid operator sequence '++'
Line 2, Column 9: Invalid operator sequence '**'
Line 2, Column 37: Invalid operator sequence '**'
Line 3, Column 5: Invalid operator sequence '&&'
Line 3, Column 7: Invalid operator sequence '||'
Line 4, Column 8: Unknown symbol '.'
Line 5, Column 9: Unknown symbol '$'
```

6. Conclusiones

El proyecto en si es un proceso largo y algo complicado debido a los requisitos del mismo además de que busca generar una cierta idea de todo lo que sucede detrás del monitor para poder comprender que significan esos pulsos eléctricos que generan los sistemas electrónicos y la importancia del orden que deben de seguir los mismos para que no se pierda el contexto de lo que se quiere enviar.

7. Referencias

Parr, T. J., & Quong, R. W. (1995). ANTLR: A predicated-LL (k) parser generator. Software: Practice and Experience, 25(7), 789-810.

Parr, T., Wells, P., Klaren, R., Craymer, L., Coker, J., Stanchfield, S., ... & Flack, C. (2004). What's ANTLR.